2026-6-7 C++哈希表

- 1.哈希的概念
  - 1.1 直接定值法
  - 2.哈希函数
  - 2.1 除法散列法/除留余数法
  - 2.2 乘法散列法
  - 2.3 全域散列法
- 3. 处理哈希冲突
  - 3.1 开放定址法
    - 3.1.1 线性探测
    - 3.1.2 二次探测
    - 3.1.3 双重散列
- 3.2 开放地址法代码实现
  - 3.2.1 哈希表结构定义
  - 3.2.2 key不能取模的问题
  - 3.2.3 Insert插入
  - 3.2.4 Find查找
  - 3.2.5 Erase删除
    - 3.3 链地址法
      - 3.3.1 哈希桶的实现
      - 3.3.2 哈希桶结构的定义
      - 3.3.3 Insert实现
      - 3.2.4 Find查找
      - 3.2.5 Erase删除

1.哈希的概念

哈希(hash)即散列，哈希是音译的，散列是形译的，是一种数据的组织形式。有散乱排列的意思。哈希的本质是通过哈希函数把关键字Key跟存储位置建立一个映射关系，查找时通过这个哈希函数计算出key存储位置，进行快速查找。

1.1 直接定值法

适用于关键字范围比较集中，本质上就是用关键字计算出一个绝对位置与相对位置，直接定址法简单且高效，每个关键字的值直接就是存储位置的下标，这个整体就是一个计数排序的思想。(这里不过多赘述)。

2.哈希函数

哈希函数直白点说就是：数据在哈希表中的映射方式。一个好的哈希函数应该让N个关键字被等概率的均匀散列分布到哈希表的M个空间中。但是在实际设计中确很难实现，但是我们尽量往这个方向去设计。

2.1 除法散列法/除留余数法

1. 除法散列法顾名思义也叫除留余数法：即假设哈希表的大小为M，那么通过K除以M的余数作为映射位置的下标：哈希函数就是：h(key)=key%M。
2. 当使用除法散列的时候要尽量避免M为某些值，如2^x或者10^x等，这样就相当于保留k的后X位，那如果两个数的后X位相同，则计算出的值就会相同。
3. 当使用除法散列法的时候，建议M取不太接近2的整数次幂的一个质数（素数：只有一和它本身两个因子的数）

2.2 乘法散列法

1. 乘法散列法对M的大小没有要求，其大体的思路是：用关键字K乘上常数A（0<A<1），并抽出A * K的小数部分，再用M乘以K * A的小数部分，再向下取整。
2. h(key)=floor(M * ((A * K) % 1.0))，其中floor是向下取整，这里比较重要的是A的值应该如何设定。（Knuth认为A=0.618033987...黄金分割点比较好）

2.3 全域散列法

1. 如果存在一个恶意的对手，他针对我们提供的散列函数特意构造出一个发生严重冲突的数据集：例如让所有关键字都落入一个位置中。这种情况是可以存在的，只要散列函数是公开且确定的，就可以实现此攻击。我们的解决办法就是给散列函数增加随机性，让攻击者无法找到确定的可以找到最坏情况的数据。
2. $h_{a,b}(k) = ((ak + b) \bmod p) \bmod m$，P需要选一个足够大的质数，a可以随机在[1,p-1]之间选择任意整数，b可以随机选[0,P-1]之间的任意整数，这些函数构成了一个P * (P-1)组全域散列函数。
3. 需要注意的是每次初始化哈希表的时候，随机选取全域散列函数组中的一个散列函数使用，后续增删改查都固定使用这个散列函数，否则每次哈希都是随机选一个散列函数，不然插入的是一个散列函数，查找又是另一个散列函数，就会导致找不到插入的key了。

3. 处理哈希冲突

实践中哈希表一般选择除法散列法作为哈希函数，当然哈希表无论选择什么哈希函数也都避免不了冲突，插入数据时，我们应该如何解决冲突？主流方式有两种：开放地址法和链地址法。

3.1 开放定址法

所有元素都放到哈希表中，当一个关键字key用哈希函数计算出的位置冲突了，则按照某种规则找到一个没有存储数据的位置进行存储，开放地址法的负载因子一定小于1。

3.1.1 线性探测

1. 从发生冲突的位置开始，依次向后探测，直到找到第一个空位置为止，如果探测到表尾就回到表头的位置。
2. h(key)=hash0=key%M,hash0位置冲突了，则线性探测公式为：hc(key,i)=hashi=(hash0+i)%,M,i={1,2,3,...,M-1}，因为负载因子小于1，则最多探测M-1次，一定能找到一个存储K的位置。
3. 线性探测的位置连续冲突，就会出现很多的数据去争夺同一个位置的情况，这种现象叫做群现象，二次探测可以在一定程度上改善上述问题。

3.1.2 二次探测

1. 从发生冲突的位置开始，依次左右按照二次方跳跃探测，直到下一个没有存储数据的位置为止，如果往右走到哈希表表尾，则回绕到哈希表头；如果往左走到哈希表头则发生回绕。
2. 二次探测的公式为：hc(key,i)=hashi=(hash0+-i^2)%M,i={1,2,3,.....,M/2}

3.1.3 双重散列

如果第一个哈希函数计算出的值发生冲突，使用第二个哈希函数计算出一个跟key相关的偏移量，不断向后探测，直到寻找到下一个没有存储数据的位置为止。

3.2 开放地址法代码实现

定义枚举类型标识哈希表中每个存储位的状态

```
enum Status
{
    EXIST,
    EMPTY,
    DELETE
};
```

① 注意这里没有逗号

以类模板的形式自定义哈希表中的数据类型 ②

```
template<class K, class V>
struct HashData
{
    pair<K, V> _kv;
    Status _status = EMPTY;
};
```

成员变量1:pair<K,V>类型_kv
成员变量2:Status枚举类型变量_status并初始化状态为空

定义仿函数解决string类型无法比较的问题 ③

```
template<class K>
struct HashFunc
{
    size_t operator()(const K& key)
    {
        return (size_t)key;
    }
};

template<> 特化
struct HashFunc<string>
{
    // BKDR
    size_t operator()(const string& str)
    {
        size_t hash = 0; 重新计算哈希值
        for (auto ch : str)
        {
            hash += ch;
            hash *= 131;
        }
        return hash;
    }
};
```

1.遍历每一位字符，将它们的ASCII值相加
2.相加后为了减少冲突每一次都乘一个131

定义哈希表的功能和特性：

```
template<class K, class V, class Hash = HashFunc<K>>
class HashTable
{
public:
    HashTable()
        :_tables(__stl_next_prime(1))
        ,_n(0)
    {}
private:
    std::vector<HashData<K, V>> _tables;
    size_t _n = 0;   // 有效数据个数
};
```

HashTable

声明一个vector:

_tables

HashData <K,V>

声明一个_n用于存储存入表中的数据

质数表

```
inline unsigned long __stl_next_prime(unsigned long n)
{
    // Note: assumes long is at least 32 bits.
    static const int __stl_num_primes = 28; 定义表大小为28
    static const unsigned long __stl_prime_list[__stl_num_primes] =
    {
        53, 97, 193, 389, 769,
        1543, 3079, 6151, 12289, 24593,
        49157, 98317, 196613, 393241, 786433,
        1572869, 3145739, 6291469, 12582917, 25165843,
        50331653, 100663319, 201326611, 402653189, 805306457,
        1610612741, 3221225473, 4294967291
    };
    const unsigned long* first = __stl_prime_list; 首地址
    const unsigned long* last = __stl_prime_list + __stl_num_primes; 尾地址
    const unsigned long* pos = lower_bound(first, last, n); 目标位置
    return pos == last ? *(last - 1) : *pos;
}
```

这是一个看起来很恐怖但是很简单的初始化

内联质数表：
为了保持哈希表的大小是一个质数，我们使用sgi版本的哈希表使用方法，这个表中存储了一些近似2倍的质数。

插入

```
bool Insert(const pair<K, V>& kv)
{
    if (Find(kv.first))
        return false;

    // 负载因子 >= 0.7 就扩容
    if ((double)_n / _tables.size() >= 0.7) 求vector的大小
    {
        HashTable<K, V, Hash> newHT;
        newHT._tables.resize(__stl_next_prime(_tables.size()+1));
        // 遍历旧表将所有值映射到新表
        for (auto& data : _tables)
        {
            if (data._status == EXIST)
            {
                newHT.Insert(data._kv);将_status中的值重新插入newHT中
            }
        }
        _tables.swap(newHT._tables);
    }
    Hash hs;
    size_t hash0 = hs(kv.first) % _tables.size();
    size_t hashi = hash0;
    size_t i = 1;
    // 线性探测
    while (_tables[hashi]._status == EXIST)
    {
        hashi = (hash0 + i) % _tables.size(); 便于回绕
        ++i;
    }
    _tables[hashi]._kv = kv;
    _tables[hashi]._status = EXIST;
    ++_n;
    return true;
}
```

HashTable中的_tables与newTable中的_tables做交换，再让HashTable中的_tables取代newTable中的_tables

处理数据保证其可以进行对比

找到要存储位置，将数据存入并改动其状态

数目+1

② 意图：创建一个新的哈希表对象 → 让新表分配好空间 → 遍历旧表，直接调用已有的 Insert 函数将元素插入新表 → 交换新旧表。
优点：复用了已经调试好的 Insert 逻辑（包括哈希计算、冲突处理等），代码更简洁、更可靠。

```
//std::vector<HashData<K, V>> newtables(_tables.size()*2);
//// 遍历旧表将所有值映射到新表
//for (auto& data : _tables)
//{
//  if (data._status == EXIST)
//  {
//      //...
//  }
//}
//_tables.swap(newtables);
```

① 意图：手动创建新表 → 手动遍历旧表 → 手动重新计算每个元素在新表中的位置 → 手动插入 → 交换新旧表。
缺点：需要手动实现哈希函数重映射、处理冲突等逻辑，代码量较大且容易出错。

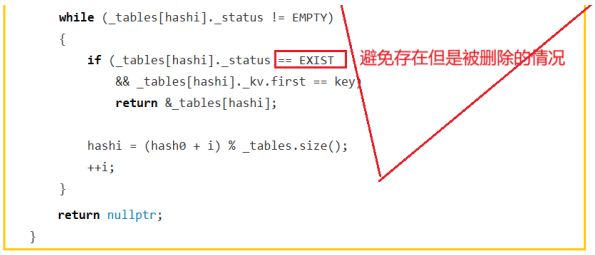
查找

```
HashData<K, V>* Find(const K& key) 这里要返回的是地址
{
    Hash hs;
    size_t hash0 = hs(key) % _tables.size();
    size_t hashi = hash0;
    size_t i = 1;
    // 线性探测
```

和上面一样，数据一定要在仿函数里面过一遍

删除

```
bool Erase(const K& key)
{
    auto* ptr = Find(key); HashData<K,V>*
    if (ptr)
    {
        ptr->_status = DELETE;
        --_n;
```





3.2.1 哈希表结构定义

枚举类型定义状态：

```
enum Status
{
    EXIST,
    EMPTY,
    DELETE
};
```

定义节点结构与内容：

```
template<class K, class V>
struct HashData
{
    pair<K, V> _kv;
    Status _status = EMPTY;
};
```

定义类框架：

```
template<class K, class V, class Hash = HashFunc<K>>
class HashTable
{
public:
    HashTable()
        :_tables(__stl_next_prime(1))
        ,_n(0)
    {}
    private:
    std::vector<HashData<K, V>> _tables;
    size_t _n = 0;    // 有效数据个数
};
```

3.2.2 key不能取模的问题

利用仿函数解决key不能取模的问题：

```
template<class K>
struct HashFunc
{
    size_t operator()(const K& key)
    {
        return (size_t)key;
    }
};
```

特化该仿函数用于处理string无法取模的问题：

```
template<>
struct HashFunc<string>
{
    // BKDR
    size_t operator()(const string& str)
    {
        size_t hash = 0;
        for (auto ch : str)
        {
            hash += ch;
            hash *= 131;
        }
        return hash;
    }
};
```

3.2.3 Insert插入

1. 若负载因子大于0.7就扩容
2. 直接对该对象进行复用，重新实例化出一个对象
3. 遍历旧空间之中的数据，将数据重新映射到新表，并与this指针进行交换
4. 不需要开辟空间或者空间开辟完成后就对即将插入的数据进行映射，如果映射到的位置以及存在数据，则对其进行线性探测，直到找到不为空的位置

```
bool Insert(const pair<K, V>& kv)
    {
        if (Find(kv.first))
            return false;
        // 负载因子 >= 0.7 就扩容
        if ((double)_n / _tables.size() >= 0.7)
        {
            HashTable<K, V, Hash> newHT;
            newHT._tables.resize(__stl_next_prime(_tables.size()+1));
            // 遍历旧表将所有值映射到新表
            for (auto& data : _tables)
            {
                if (data._status == EXIST)
                {
                    newHT.Insert(data._kv);
                }
            }
            _tables.swap(newHT._tables);
        }
        Hash hs;
        size_t hash0 = hs(kv.first) % _tables.size();
        size_t hashi = hash0;
        size_t i = 1;
        // 线性探测
        while (_tables[hashi]._status == EXIST)
        {
            hashi = (hash0 + i) % _tables.size();
            ++i;
        }
        _tables[hashi]._kv = kv;
        _tables[hashi]._status = EXIST;
        ++_n;
        return true;
    }
```

3.2.4 Find查找

若不为映射的位置不为空则需要确保该位置的状态只能为存在，因为如果该数据被删除了之后数据依旧在，只是状态改变了所以要确保状态。

```
HashData<K, V>* Find(const K& key)
    {
        Hash hs;
```

```
        size_t hash0 = hs(key) % _tables.size();
        size_t hashi = hash0;
        size_t i = 1;
        // 线性探测
        while (_tables[hashi]._status != EMPTY)
        {
            if (_tables[hashi]._status == EXIST
                && _tables[hashi]._kv.first == key)
                return &_tables[hashi];
            hashi = (hash0 + i) % _tables.size();
            ++i;
        }
        return nullptr;
    }
```

3.2.5 Erase删除

整体思路就是找到要删除数据的位置，将其状态置为DELETE即可。

```
bool Erase(const K& key)
    {
        auto* ptr = Find(key);
        if (ptr)
        {
            ptr->_status = DELETE;
            --_n;
            return true;
        }
        else
        {
            return false;
        }
    }
```

3.3 链地址法

1. 哈希表中存储一个指针，没有数据映射到这个位置时，这个指针为空，有多个数据映射到这个位置时，我们把这些冲突的数据链接成一个链表，挂在哈希表的这个位置下面，链地址法又叫拉链法或者哈希桶。
2. 链地址法的负载因子可以大于1，负载因子越大，哈希冲突的概率就越高，空间利用率越高。这里我们采用大于1就扩容的方式。

极端场景：

某个桶特别长，可以考虑全域散列的方式。但是在偶然情况下，如果某个桶非常长，查找效率非常低怎么办。Java中将其改为了红黑树。

3.3.1 哈希桶的实现

哈希桶自定义节点结构：①

```
template<class K, class V>
struct HashNode
{
    pair<K, V> _kv;
    HashNode<K, V>* _next;

    HashNode(const pair<K, V>& kv)
        :_kv(kv)
        ,_next(nullptr)
    {}
};
```

和list一样

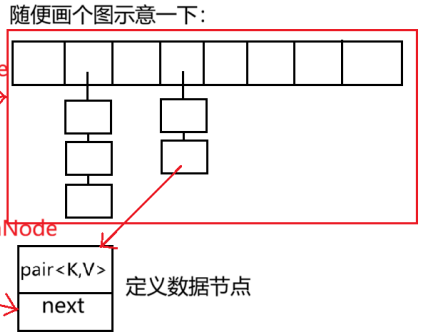
哈希桶结构：定义哈希表，并把节点放进去

```
template<class K, class V, class Hash = HashFunc<K>>
class HashTable
{
    typedef HashNode<K, V> Node;
public:
    HashTable()
        :_tables(__stl_next_prime(1), nullptr)
        ,_n(0)
    {}
private:
    //vector<list<pair<K, V>>> _tables;
    vector<Node*> _tables;
    size_t _n = 0;  // 实际存储的数据个数
};
```

指针数组，初始化指针大小并将地址置为nullptr

本质也就是vector套了一个list

随便画个图示意一下：②



插入函数

```
bool Insert(const pair<K, V>& kv)
{
    if (Find(kv.first))
        return false;
    Hash hs;
    // 负载因子==1扩容
    if (_n == _tables.size())
    {
      vector<Node*> newtables(__stl_next_prime(_tables.size()+1));
      for (size_t i = 0; i < _tables.size(); i++)
      {
          Node* cur = _tables[i];
          // 当前桶的节点重新映射挂到新表
          while (cur)
          {
              Node* next = cur->_next;

              // 插入到新表
              size_t hashi = hs(cur->_kv.first) % newtables.size();
              cur->_next = newtables[hashi];
              newtables[hashi] = cur;
              cur = next;
          }

          _tables[i] = nullptr;
      }
      _tables.swap(newtables);
    }
    size_t hashi = hs(kv.first) % _tables.size();
    // 头插
    Node* newNode = new Node(kv);
    newNode->_next = _tables[hashi];
    _tables[hashi] = newNode;
    ++_n;
    return true;
}
```

调用仿函数

获得指针数组的大小

取下一个质数为数组的大小

定义指针遍历整个哈希表

cur不为空

说明该桶不为空，进入桶中遍历

重新将数据映射到newtables中

需要特别注意这里的操作：

这里并没有像开放地址法一样复用新对象的Insert,因为要复用Insert的话每个节点都需要重新开辟一个空间就很浪费。这里直接把原本的节点“搬”过来再与this指针交换一下。

不需要重新开辟空间，就直接映射就好，映射之前把数据给hs处理一下

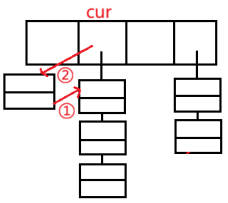
插入需要重新开辟空间

数量++

```
//HashTable<K, V> newHT;
//newHT._tables.resize(_tables.size()*2);
//// 遍历旧表将所有值映射到新表
//for (auto cur : _tables)
//{
//  while (cur)
//  {
//      newHT.Insert(cur->_kv);
//      cur = cur->_next;
//  }
//}
//_tables.swap(newHT._tables);
```

要为每个数据重复开辟空间，资源浪费直接用现成即可

插入过程示例：



析构函数

```
~HashTable()
{
    for (size_t i = 0; i < _tables.size(); i++)
    {
        Node* cur = _tables[i];
        // 当前桶的节点重新映射挂到新表
        while (cur)
        {
            Node* next = cur->_next;
            delete cur;
            cur = next;
        }

        _tables[i] = nullptr;
    }
}
```

遍历整个哈希表

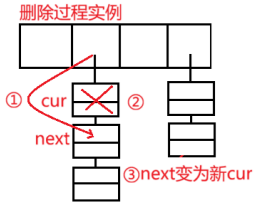
定义该节点用于遍历

cur不为空则深入处理

逐一删除还要存储一个next记录下一个节点的位置

删除后置空

删除过程实例



查找

```
Node* Find(const K& key)
{
    Hash hs;
    size_t hashi = hs(key) % _tables.size();
    Node* cur = _tables[hashi];
    while (cur)
    {
        if (cur->_kv.first == key)
            return cur;
```

先找到映射位置再往桶内寻找

不为空继续往下寻找

删除

```
bool Erase(const K& key)
{
    Hash hs;
    size_t hashi = hs(key) % _tables.size();
    Node* prev = nullptr;
    Node* cur = _tables[hashi];
    while (cur)
    {
        if (cur->_kv.first == key)
        {
            if (prev == nullptr)
```

删除也需要先找到映射位置

标记cur向下寻找

该情况说明需要删除的节点在该桶的最上



3.3.2 哈希桶结构的定义

哈希桶节点结构的定义：

```
template<class K, class V>
    struct HashNode
    {
        pair<K, V> _kv;
        HashNode<K, V>* _next;
        HashNode(const pair<K, V>& kv)
            :_kv(kv)
            ,_next(nullptr)
        {}
    };
```

哈希桶类结构的定义：

```
template<class K, class V, class Hash = HashFunc<K>>
    class HashTable
    {
        typedef HashNode<K, V> Node;
    public:
        HashTable()
            :_tables(__stl_next_prime(1), nullptr)
            ,_n(0)
        {}
        private:
        //vector<list<pair<K, V>>> _tables;
        vector<Node*> _tables;
        size_t _n = 0;  // 实际存储的数据个数
    };
```

哈希桶还需要自己实现一个析构函数：

```
~HashTable()
        {
            for (size_t i = 0; i < _tables.size(); i++)
            {
                Node* cur = _tables[i];
                // 当前桶的节点重新映射挂到新表
                while (cur)
                {
                    Node* next = cur->_next;
                    delete cur;
                    cur = next;
                }
                _tables[i] = nullptr;
            }
        }
```

哈希桶也需要解决数据取模的问题，这个和开放地址法的内容相同，这里就不过多赘述了。
补充一个拷贝构造：

```
HashTable(const HashTable& other)
    : _n(other._n)
{
    _tables.resize(other._tables.size(), nullptr);
    for (size_t i = 0; i < other._tables.size(); ++i)
    {
        Node* cur = other._tables[i];
        while (cur)
        {
            Node* copy = new Node(cur->_kv);
            // 头插到对应位置
            size_t hash = hs(copy->_kv.first) % _tables.size();
            copy->_next = _tables[hash];
            _tables[hash] = copy;
            cur = cur->_next;
        }
    }
}
```

3.3.3 Insert实现

1. 直接计算出映射位置

2. 头插还是尾插都行，我们这里从头部插，第一个结点的位置在表里，我插入成为第一个结点
3. 桶是空不需要单独处理，其过程是一模一样的
4. 哈希桶的查找插入其的效率会抖动，正常效率都很高，但是某一次可能会很慢（扩容的时候）我们要看平均时间，但是主要影响在于扩容的那个时候
5. 注意要去一下重

扩容：
负载因子等于1，就需要扩容。扩容就涉及到重新映射

1. 遍历旧表重新映射到新表里，遍历的时候是重新创建结点，还需要重新实现析构，会造成严重的空间浪费
2. 能不能直接把源节点给直接拿下来不为每个结点重新开辟空间呢？因为源链表的结点越多拷贝析构的值就会极大的浪费空间。如果我们复用插入必然会出现上述情况。
3. 总体思路就是使用for循环,循环遍历重新将结点插入到新表。
4. 为什么不使用范围for,因为范围for本质是动指针，然而我们的思路结点的指针不变，原表的指针也不变，全部拿走后要将结点置空。
5. 最后交换newtable。
6. 构造的时候最开始最好开点空间或者一开始就检查一下
   实例代码如下：

```
bool Insert(const pair<K, V>& kv)
        {
            if (Find(kv.first))
                return false;
            Hash hs;
            // 负载因子==1扩容
            if (_n == _tables.size())
            {
                vector<Node*> newtables(__stl_next_prime(_tables.size()+1));
                for (size_t i = 0; i < _tables.size(); i++)
                {
                    Node* cur = _tables[i];
                    // 当前桶的节点重新映射挂到新表
                    while (cur)
                    {
                        Node* next = cur->_next;
                        // 插入到新表
                        size_t hashi = hs(cur->_kv.first) %
newtables.size();
                        cur->_next = newtables[hashi];
```

```
                    newtables[hashi] = cur;
                    cur = next;
                }
                _tables[i] = nullptr;
            }
            _tables.swap(newtables);
        }
        size_t hashi = hs(kv.first) % _tables.size();
        // 头插
        Node* newNode = new Node(kv);
        newNode->_next = _tables[hashi];
        _tables[hashi] = newNode;
        ++_n;
        return true;
    }
```

3.2.4 Find查找

直接就是转化为了链表的查找
整体思路就是，顺着桶找找不到就是下一个桶。

```
Node* Find(const K& key)
    {
        Hash hs;
        size_t hashi = hs(key) % _tables.size();
        Node* cur = _tables[hashi];
        while (cur)
        {
            if (cur->_kv.first == key)
                return cur;
            cur = cur->_next;
        }
        return nullptr;
    }
```

3.2.5 Erase删除

删除不能复用find,因为这是单链表，可能没有办法找到其上一个结点。得需要找的结点来一个跟班。但是要防止cur结点在桶内只有它自己，没有前一个结点咋搞，这里要单独处理一下。
即判断pre `==` nullprt的情况

```
bool Erase(const K& key)
        {
            Hash hs;
            size_t hashi = hs(key) % _tables.size();
            Node* prev = nullptr;
            Node* cur = _tables[hashi];
            while (cur)
            {
                if (cur->_kv.first == key)
                {
                    if (prev == nullptr)
                    {
                        _tables[hashi] = cur->_next;
                    }
                    else
                    {
                        prev->_next = cur->_next;
                    }
                    delete cur;
                    return true;
                }
                prev = cur;
                cur = cur->_next;
            }
            return false;
        }
```